Автономная некоммерческая организация высшего образования
«Университет Иннополис»

ВЫПУСКНАЯ КВАЛИФИКАЦИОННАЯ РАБОТА
(БАКАЛАВРСКАЯ РАБОТА)
по направлению подготовки
09.03.01 - «Информатика и вычислительная техника»

GRADUATION THESIS
(BACHELOR'S GRADUATION THESIS)
Field of Study
09.03.01 – "Computer Science"

Направленность (профиль) образовательной программы
«Анализ данных и искусственный интеллект»

Area of Specialization / Academic Program Title:
"Data Analysis and Artificial Intelligence"

Тема / Topic	**Надёжный код: сравнительный анализ методов оценки неопределённости для кодовых LLM / Code You Can Trust: Benchmarking Uncertainty Quantification Methods for Code LLMs**	
Работу выполнил / Thesis is executed by	**Софья Ткаченко / Sofia Tkachenko**	подпись / signature
Руководитель выпускной квалификационной работы / Supervisor of Graduation Thesis	**Владимир Иванов / Vladimir Ivanov**	подпись / signature
Консультанты / Consultants		подпись / signature

Иннополис, Innopolis, 2026

Contents

List of Tables

List of Figures

Abstract

Large language models for code generation can produce confident but incorrect outputs. Uncertainty quantification (UQ) methods aim to detect such failures, but current evaluations are difficult to compare across models, benchmarks, and metrics. This thesis evaluates thirteen unsupervised UQ methods on the HumanEval [3] benchmark using three open-source code models: DeepSeek-Coder-6.7B, Qwen2.5-Coder-7B , and DeepSeek-Coder-1.3B [4], [5]. The methods include LM-Polygraph [2] estimators and execution-based approaches such as functional and symbolic clustering [6], [7]. A shared greedy decoding setup ensures that results reflect uncertainty estimation quality. The results show that no single method is best across all models. By PR-AUC, the leading methods vary by model: functional clustering for DeepSeek-6.7B, CCP and MSP for Qwen, and ROUGE-L and CCP for DeepSeek-1.3B. ROUGE-L and CCP remain consistently strong across all models. By PRR, a revised symbolic clustering method ranks among the top approaches on larger models. Functional clustering performs well on the largest model but poorly on the smallest, suggesting sensitivity to model scale and output diversity.

Chapter 1

Introduction

Large Language Models (LLMs) are widely used to generate code. The Stack Overflow Developer Surveys [1] show LLM adoption rising from 43.8% to 78.5% between 2023 and 2025. Yet the share of developers who distrust AI accuracy grew from 27.2% to 45.7% over the same period (Fig. 1.1).

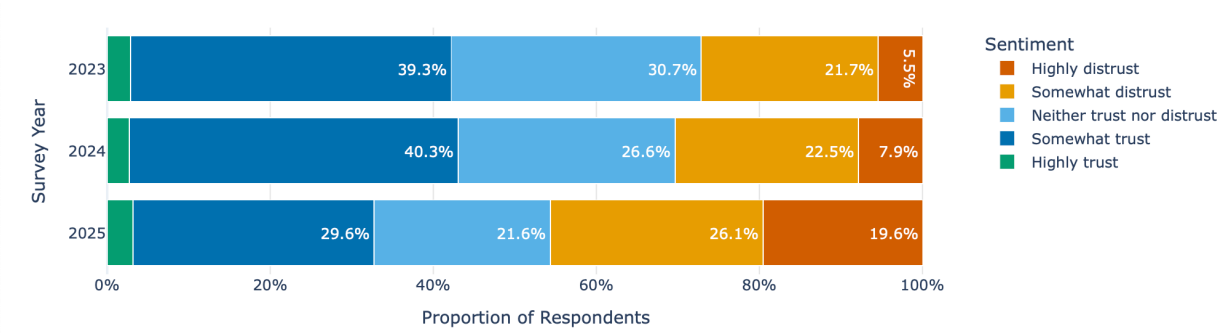

Fig. 1.1. Developer Trust in AI Accuracy (2023-2025) [1]

The root cause of distrust is LLM hallucinations: models produce plausible but incorrect code. Uncertainty Quantification (UQ) methods can detect when a model is likely wrong and flag suspicious outputs before they reach production. However, most existing UQ methods were developed for natural language generation and do not transfer well to code. Recent studies [6], [7] show that standard text-similarity and token-probability methods have no statistically significant correlation with code correctness. In contrast, methods that run the generated programs perform far better.

For natural language tasks, such as question answering, UQ methods are systematically compared using the LM-Polygraph benchmark [2]. Currently, no such benchmark exists for code. Moreover, code UQ research is mostly incomparable: different studies use different models, tasks, and metrics.

This thesis aims to fill that gap with two objectives:

- **First**, to evaluate the best-performing unsupervised LM-Polygraph methods on code generation and determine how well they transfer from natural language.
- **Second**, to compare them against code-specific execution-based methods (functional clustering [6] and symbolic clustering [7]) on a shared benchmark.

The goal is to obtain a quantitative answer to which UQ strategies work best for code.

Chapter 2

Background

2.1 Hallucinations

Ji *et al.* [8] defined hallucinations as "generated content that is nonsensical or unfaithful to the provided source content". LLMs are prone to hallucinations because they predict the next most probable token. Training rewards plausible output for any prompt, even when the model lacks the necessary information. The model is penalized if it says "I do not know" or gives an incomplete answer. As a result, the model learns to produce complete answers regardless of confidence, leading to hallucinations.

The model's knowledge is a compressed representation of its training data. Reconstruction from compressed patterns introduces errors and false associations. Additionally, the training data itself can contain factual errors, outdated information, and contradictions, which the model can reproduce.

Code generation has its own hallucination problems. CodeMirage [9], Hallu-Code [10], CodeHalu [11], and Collu-Bench [12] show that LLMs frequently generate plausible-looking code with hard-to-detect defects: logical flaws, security vulnerabilities, unreachable code, or calls to non-existent libraries and functions [9].

2.2 Uncertainty and Confidence

There are two related concepts that are important for understanding and addressing hallucinations in LLMs: uncertainty and confidence.

- *Uncertainty* is a property of the input. Given a prompt x, the model's predicted distribution $P(Y|X = x)$ may be spread over many possible outputs or concentrated on a few. A vague prompt like "write a sort function" has many valid implementations, so the distribution is wide. A more precise prompt like "write a function that returns the sum of two integers" leaves less room for variation. Uncertainty depends on x only, not on any particular output [13].
- *Confidence* is a property of a specific prediction. Given input x and a generated output y, confidence measures how likely y is to be correct. A model can be uncertain about a prompt (many possible outputs) while still being confident in the one it chose [13].

However, in practice, researchers often use "uncertainty" and "confidence" interchangeably. When measuring uncertainty for each generated output, the measurement is technically confidence. This thesis uses "uncertainty" to refer to the model's confidence in its output, following common usage. The thesis also converts confidence scores to uncertainty scores by inverting them ($\text{uncertainty} = 1 - \text{confidence}$) to be consistent with LM-Polygraph.

2.2.1 Types of Uncertainty

Uncertainty in LLMs comes from two sources [13]:

- *Epistemic uncertainty* (model uncertainty) appears from limitations of the model: gaps in training data or insufficient capacity. A model that never saw a particular type of problem during training will be epistemically uncertain about it. This type can, in principle, be reduced with more data or better training.
- *Aleatoric uncertainty* (data uncertainty) comes from the inherent ambiguity of the task. Some problems have multiple correct solutions (e.g., "write a sorting function" can be solved with quicksort, mergesort, or insertion sort). This type

of uncertainty cannot be reduced by improving the model because the ambiguity is in the problem.

Separating total uncertainty into these components is complex and, in practice, usually unnecessary [13]. This thesis measures total uncertainty without decomposing it.

2.3 Uncertainty Quantification

Uncertainty Quantification (UQ) is the set of methods that assign an uncertainty score to a model's output, so that highly uncertain generations can be rejected or handed off to a human for review. The idea comes from selective classification, where a classifier can abstain on inputs it cannot answer reliably rather than risk a wrong prediction [14], [15]. Abstention is important in domains such as banking, healthcare, and law, where a deferred decision is far preferable to a wrong one. The same logic applies to code: a flagged hallucination can be reviewed before it reaches production.

UQ methods first appeared in classification and regression [16], [17], based on information theory and Bayesian modeling [18]. They were later adapted to encoder language models such as BERT [19]. The scale and free-form output of modern text-generating LLMs required new approaches [20], leading to the current line of work on UQ for text generation [21]. UQ also underpins related tasks such as out-of-distribution detection [22], [23], defense against adversarial attacks [24], and active learning [16].

UQ methods differ in the access to the model they require [13]. **White-box** methods read internal signals such as logits or attention weights. **Black-box** methods use only the generated text, which makes them applicable to proprietary API-based models.

Chapter 3

Literature Review

3.1 UQ Method Categories

The LM-Polygraph benchmark [2] evaluates 29 UQ methods on natural language generation and organizes them into four categories:

- **Information-Theoretic:** compute uncertainty from the model's token-level probability distribution. Examples include Maximum Sequence Probability, Perplexity, and Mean Token Entropy [25].
- **Sample Diversity:** generate multiple completions for the same prompt and measure their disagreement. Consistent outputs indicate higher confidence. Examples include Semantic Entropy [21], Degree Matrix, and Sum of Eigenvalues of the Graph Laplacian [13].
- **Density-based:** compare a generated output's embedding against a reference distribution of correct examples. Outputs far from the distribution are flagged as uncertain. Examples are Mahalanobis distance (MD) [26] and Robust density estimation (RDE) [27].
- **Reflexive:** ask the model to evaluate its own confidence directly (e.g., "Are you certain about your last response?"). The $p(\mathrm{True})$ method [28] is an example.

Each category comes with its strengths and weaknesses (according to [2]):

TABLE 1

Comparison of Uncertainty Quantification Method Categories

Category	Access	Compute	Strengths	Weaknesses
Information-Theoretic	White-Box	Low	Fast, theoretically grounded, provides token-level scores	Requires internal model access. Can be naive to semantic meaning
Sample Diversity	Black-Box	High	Model-agnostic. Captures semantic uncertainty	Requires generating many outputs. Clustering can be complex and slow
Density-Based	Black-Box	Medium	Good at detecting outputs that are structurally or stylistically unusual	Requires a high-quality reference dataset: performance depends heavily on the embedding space
Reflexive	Black-Box	Low	Simple to implement	Highly unreliable: models are often overconfident and poor at self-assessment

3.2 Code Hallucination Taxonomies

Several recent works classify the types of errors that occur when large language models generate code. While each proposes a distinct taxonomy, their categories overlap considerably. Understanding these taxonomies is important for uncertainty

quantification, as different types of hallucinations are caught by different detection strategies.

CodeMirage [9] analyzed 1,137 Python code snippets generated by GPT-3.5 on HumanEval and MBPP benchmarks. The authors annotated each snippet with one of five defect types: logical errors (the code runs but solves the wrong problem), syntactic incorrectness (the code does not compile), dead or unreachable code (segments that serve no purpose), robustness issues (the code fails on edge cases or raises exceptions), and security vulnerabilities (the code has exploitable flaws or memory leaks). These defect types appeared in roughly equal proportions in their dataset.

HalluCode [10] classifies hallucinations based on what the code conflicts with, rather than the technical nature of the defect. This taxonomy, developed for Python and Java tasks, defines three broad categories with twelve subcategories. Requirement conflicts (39.6%) cover cases where the code does something other than what was specified, with behavior conflicts (35.4%) as the largest subcategory. Knowledge hallucinations (34.9%) capture cases where the code contradicts real-world knowledge, such as incorrect use of library APIs (25.99%) or algorithmic conventions. Code inconsistency (25.5%) refers to internal contradictions, like undefined variables or redundant statements. The study found that model-related factors account for the majority (80.86%) of hallucinations and that larger models tend to hallucinate less.

CodeHalu [11] organizes errors by their source. Mapping hallucinations arise when the model incorrectly maps between the problem description and code variables or functions. Naming hallucinations involve the use of incorrect identifiers. Resource hallucinations refer to references to non-existent libraries or functions. Logic hallucinations involve flawed algorithmic reasoning. These four categories are further divided into eight subcategories and are validated by executing the generated code.

These taxonomies overlap but emphasize different aspects of the same underlying problem. Table 2 combines them into an overview:

TABLE 2
Code Hallucination Taxonomy

Hallucination Types	Description
• Logical Error [9] • Intent Conflicting [10] • Logic Hallucinations [11]	The code is syntactically valid but fails to meet the problem's requirements or contains flawed logic
• Syntactic Incorrectness [9] • Dead or Unreachable Code [9] • Naming Hallucinations [11] • Context Deviation [10]	The code is malformed, uses wrong identifiers, contains non-functional or repetitive segments, or cannot be compiled
• Robustness Issue [9]	The code compiles but fails at runtime on edge cases or lacks exception handling
• Security Vulnerabilities [9] • Knowledge Conflicting [10] • Mapping Hallucinations [11] • Resource Hallucinations [11]	The code misuses variables, external APIs, or libraries, or calls non-existent resources, leading to errors or security flaws

As Table 2 shows, code hallucinations differ from general text generation errors. Methods from natural language generation (NLG) require adaptation to work on code.

3.3 Evaluating Uncertainty Quantification Methods

Measuring UQ performance is challenging. Literature uses several approaches:

- Rank correlation (Spearman's ρ) between uncertainty scores and output quality metrics such as ROUGE or BLEU [25], [29]. These metrics reveal little about practical performance, and n-gram metrics often miss semantic quality.
- Binary classification (correct vs. incorrect output) evaluated with AUROC or PR-AUC [2], [29], [30]. This approach requires an arbitrary correctness threshold, making cross-study comparison difficult.

This thesis uses both PR-AUC and the Prediction Rejection Ratio (PRR) [31], with PR-AUC as the primary metric. PRR is the standard in LM-Polygraph [2] and earlier work [20]. Both are defined in Chapter 4.4.

3.4 Uncertainty Quantification for Code

UnCert-CoT [32] uses token entropy to switch from direct generation to Chain-of-Thought reasoning when uncertainty is high. AdaDec [33] uses token-level entropy to pause decoding and rerank candidate tokens.

Token-level signals often fail for code because syntactically different programs can do the same thing. Recent work clusters outputs by behavior rather than text.

Ravuri and Amarasinghe [6] propose functional clustering, arguing that text embeddings miss small errors, such as a swapped operator, that can break code. Their method generates programs and test inputs, runs them in a sandbox, and groups programs with identical input-output behavior. The size of the largest group becomes the confidence score. On LiveCodeBench, this reduced error rates from about 65% to 2%, while token-probability scores could not reliably separate correct from incorrect outputs.

Sharma and David [7] propose symbolic clustering, which goes beyond unit tests by treating inputs as abstract symbols and generating code execution traces via symbolic execution. They group programs whose traces match. In testing several standard UQ approaches on code, they found Pearson correlations with correctness near zero: text-similarity methods reached $r = -0.04$ to -0.21 (all $p > 0.08$), and token log-probabilities $r = 0.09$ to 0.18 ($p > 0.38$), none statistically significant. Only symbolic clustering achieved meaningful correlations ($r = -0.40$ to -0.56, $p < 0.001$), with a false-positive rate below 0.02%.

Both functional and symbolic clustering are sample-diversity methods like Semantic Entropy and are computationally expensive.

Other approaches use classical code metrics as proxies: compiler feedback [34], static code quality analysis [35], and pass rates of generated unit tests [36].

Chapter 4

Methodology

4.1 Models

The evaluation uses three open-source instruction-tuned code models: **DeepSeek-Coder-6.7B-Instruct** [4], **Qwen2.5-Coder-7B-Instruct** [5], and **DeepSeek-Coder-1.3B-Instruct** [4]. The two 7B models control for parameter count across families but differ in training data and chat template format. I include the 1.3B DeepSeek-Coder variant to test whether rankings hold when model capability drops sharply (pass@1 falls from 68.9% to 50.6% on HumanEval). I tried base model variants first, but they did not consistently recognize the task as code completion.

4.2 Dataset

All experiments use **HumanEval** [3], a benchmark of 164 Python programming problems. Each problem consists of a function signature with type annotations, a docstring with example input-output pairs, and a hidden unit test suite. The model receives only the stub (signature and docstring) and must generate the function body. Correctness is determined by running the completion against the unit tests.

The prompt wraps the stub in a fenced Python block with an instruction to complete it without modifying the given code, following DeepSeek's HumanEval protocol [4]:

```
Please continue to complete the function. You are not allowed to
modify the given code and do the completion only. Please return
all completed function in a codeblock. Here is the given code to
do completion:

    ```python

    {code}

    ```
```

4.3 Uncertainty Estimation Methods

This evaluation covers only unsupervised methods, which produce uncertainty scores without any labaled calibration data. Supervised methods, such as those that fit a reference distribution of correct examples, are excluded. The reason is that training those methods is out of scope for this thesis.

Standard UQ methods from natural language generation transfer poorly to code. According to [7], text-similarity and token-probability approaches show no significant correlation with code correctness. Therefore, I pair the best-performing LM-Polygraph [2] methods with execution-based methods that assess code behavior directly.

Thirteen uncertainty scores are evaluated in total: nine from the **LM-Polygraph** [2] framework and four from two execution-based approaches, each producing two scores from the same cluster structure (cluster count and semantic entropy). Table 3 summarises them.

TABLE 3
Summary of evaluated methods

Method	Category	Access	Compute	Aux. Model
MSP	Information-Theoretic	White-box	Low	No

Method	Category	Access	Compute	Aux. Model
Perplexity [25]	Information-Theoretic	White-box	Low	No
LexSim ROUGE-L [25]	Sample Diversity	Black-box	High	No
LexSim BLEU [25]	Sample Diversity	Black-box	High	No
DegMat Jaccard [13]	Sample Diversity	Black-box	High	No
CCP [37]	Information-Theoretic	White-box	High	DeBERTa
SAR [38]	Sample Diversity	White-box	High	DeBERTa
TokenSAR	Information-Theoretic	White-box	High	DeBERTa
DegMat NLI [13]	Sample Diversity	Black-box	High	DeBERTa
Functional Clustering [6]	Execution-based	Black-box	High	No
Symbolic Clustering [7]	Execution-based	Black-box	High	CrossHair

4.3.1 Method Selection

I selected the nine LM-Polygraph methods by ranking all unsupervised estimators in the framework by mean PRR across two models (Stable LM 2 12B and Mistral 7B v0.2) in the original benchmark [2]. The top ten by mean PRR were MSP, CCP, SAR, HUQ-MD, ROUGE-L, DegMat NLI, Perplexity, DegMat Jaccard, BLEU, and TokenSAR. I excluded HUQ-MD because it is supervised: it requires a reference distribution of correct examples to compute Mahalanobis distances. The remaining nine span the information-based and sample-diversity categories. Reflexive methods did not reach the top ten.

Then the picked methods are grouped by whether they require an auxiliary NLI model. The first group (MSP, Perplexity, Lexical Similarity, DegMat-Jaccard) operates without an auxiliary model. The second group (CCP, SAR, TokenSAR,

DegMat-NLI) additionally uses DeBERTa for semantic agreement. The execution-based methods form a third group.

4.3.2 Non-NLI Methods

These methods use only token-level log-probabilities from a single greedy pass and $N = 10$ stochastic samples. No auxiliary model is required.

Maximum Sequence Probability (MSP) is the product of the greedy token probabilities: $U_{\text{MSP}} = 1 - P(y \mid x)$, where y is the generated sequence, and x is the prompt.

Perplexity [25] normalizes MSP by sequence length, so scores are comparable across outputs of different lengths:

$$U_{\text{PPL}} = \exp\left(-\frac{1}{L}\log p(x \mid y)\right) \tag{1}$$

Higher perplexity indicates higher uncertainty.

Lexical Similarity [25] methods measure consistency across samples. The mean overlap between the greedy completion and each of the N stochastic samples is computed using ROUGE-L [39] (longest common subsequence recall) and separately using BLEU [40] (n-gram precision). A model that generates diverse completions for the same problem is considered more uncertain than those whose completions are consistent.

Degree Matrix-Jaccard (DegMat-Jaccard) [13] measures consistency across all pairs of samples rather than against the greedy output. A graph is constructed with one node per sample and edge weights given by the Jaccard similarity of the two samples' token sets. The uncertainty score is derived from the spectral properties of the graph's degree matrix: similar completions give low spectral spread, diverse ones give high.

4.3.3 NLI Methods

These methods additionally pass pairs of completions through DeBERTa [41], a bidirectional transformer trained on natural language inference (NLI). Given two texts, it outputs probabilities for entailment, neutrality, and contradiction. Entailment probability serves as a proxy for semantic agreement between completions: an approximation when applied to code, but richer than token overlap.

Claim-Conditioned Probability (CCP) [37] re-weights the greedy token probabilities using NLI entailment scores from the sampled alternatives. If many samples are semantically consistent with the greedy completion, their token probabilities are upweighted, lowering the uncertainty estimate.

SAR (Semantic Answer Replacement) [38] measures how sensitive the greedy output is to token substitution. For each token, semantically equivalent alternatives from the sampled completions are identified using DeBERTa, and the change in sequence probability from the substitution is recorded. Tokens whose replacement barely shifts probability are less informative, while load-bearing tokens shift it more. High average sensitivity indicates higher uncertainty.

TokenSAR [38] uses the same per-token sensitivities as SAR but weights each by its contribution to the overall sequence probability before aggregation.

Degree Matrix-NLI (DegMat-NLI) [13] applies the same graph-based framework as DegMat-Jaccard but uses NLI entailment probability as the edge weight rather than Jaccard token overlap, capturing semantic rather than surface-level agreement.

4.3.4 Execution-Based Methods

Functional clustering and symbolic clustering use neither token probabilities nor text similarity. Instead, they generate $N = 10$ (in this experiment) stochastic completions per problem and group them by whether they produce the same behavior when executed. The assumption: a model producing behaviorally equivalent completions is more likely to be correct than one whose completions diverge.

Each method partitions the N completions into equivalence classes, from which two uncertainty scores are computed. **Cluster Count (CC)** [6], [7] is:

$$U_{\mathrm{CC}} = 1 - \frac{|C_{\max}|}{N} \tag{2}$$

where $|C_{\max}|$ is the size of the largest class. CC is zero when all completions are equivalent and approaches one when every completion is in its own class.

Semantic Entropy (SE) [7], [21] is the Shannon entropy of the cluster size distribution under a uniform prior:

$$U_{\mathrm{SE}} = -\sum_{c} \frac{|c|}{N} * \log \frac{|c|}{N} \tag{3}$$

SE is zero when all completions fall in one cluster and is maximized when they spread evenly across many. Sharma *et al.* [7] shows that CC and SE produce comparable results when clustering is accurate. Both are reported for direct comparison.

Sharma *et al.* [7] also propose Mutual Information (MI), which queries the model twice per problem with the second prompt formed by appending the first response to the original. MI is excluded here because it produces structurally different completions, which would make pass@1 incomparable across methods.

The two execution-based methods use the same CC and SE formulas and differ only in how equivalence is determined.

Functional Clustering

Proposed by [6], this method determines equivalence by running each completion on concrete test inputs and grouping completions that produce identical outputs on every input. Test inputs are generated by prompting the LLM itself given only the function signature, so the method is self-contained and works on benchmarks without ground-truth tests.

The limitation is that equivalence is tested only on a finite set of inputs. Two functions that agree on every provided input but differ elsewhere will be incorrectly merged, underestimating uncertainty.

Symbolic Clustering

Proposed by [7], this method determines equivalence using symbolic execution rather than concrete inputs. The goal is to find a counterexample (a concrete input on which two functions differ) by reasoning over all possible inputs at once.

This is done using CrossHair [42], a Python symbolic execution engine with an SMT solver. For each of the $C_N^2 = \frac{N(N-1)}{2}$ pairs of completions, CrossHair explores both functions' execution paths with symbolic inputs and asks whether any input assignment causes them to diverge. If one is found, the pair goes into separate clusters. If no counterexample is found before the timeout, the functions are declared equivalent and merged via union-find, which ensures transitivity: if $f_i \equiv f_j$ and $f_j \equiv f_k$, all three are placed in the same cluster.

The limitation is that symbolic execution is bounded: CrossHair explores paths only up to a configurable depth.

4.4 Evaluation Metrics

Each method produces a scalar uncertainty score $u_i \in \mathbb{R}$ and a binary correctness label $c_i \in \{0, 1\}$ per problem, where $c_i = 1$ if the greedy completion passes all unit tests. Three metrics are reported.

Pass@1 is the fraction of problems solved:

$$\text{pass@1} = \frac{1}{M} \sum_{i=1}^{M} c_i, \quad M = 164 \tag{4}$$

All methods share the same greedy completions, so pass@1 is identical across all thirteen scores for a given model. It shows model capability and can be used as a baseline for uncertainty estimation performance.

Prediction Rejection Ratio (PRR) [31] measures how well uncertainty scores identify incorrect predictions. Problems are ranked by decreasing uncertainty and progressively removed. At each threshold, accuracy is computed on the remaining ones. PRR is the area between this curve and the random-rejection baseline (a

horizontal line at pass@1), normalised by the area between the oracle curve (which rejects all incorrect predictions first) and the same baseline:

$$\mathrm{PRR} = \frac{\mathrm{AUC}_{\mathrm{uq}} - \mathrm{AUC}_{\mathrm{random}}}{\mathrm{AUC}_{\mathrm{oracle}} - \mathrm{AUC}_{\mathrm{random}}} \tag{5}$$

$\mathrm{PRR} = 1$ means perfect failure identification. $\mathrm{PRR} = 0$ means random ranking. Fig. 4.1 shows the relationship between these curves.

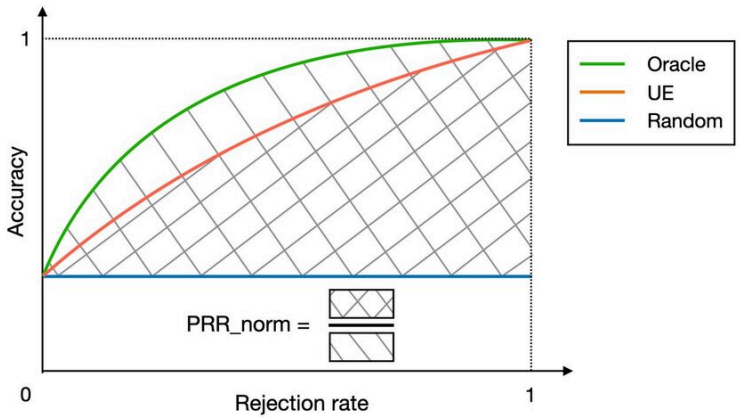

Fig. 4.1. Prediction-rejection curve explanation [2]

PR-AUC is the area under the precision-recall curve, treating failures ($c_i = 0$) as the positive class and uncertainty as the detection score. Precision is the fraction of flagged predictions that are incorrect. Recall is the fraction of all incorrect predictions that are flagged. PR-AUC aggregates this trade-off across all thresholds.

PR-AUC is the primary metric of this thesis. The intended use of code UQ is a threshold decision: a generated function is flagged for review or accepted. PR-AUC measures the precision-recall trade-off at every threshold, while PRR measures the full ranking of problems by uncertainty, independent of any specific threshold. With pass@1 of 0.689 and 0.726, failures are the minority class, where PR-AUC is preferred.

PRR is reported alongside because it is the standard metric in LM-Polygraph [2] and most natural-language UQ work, and because it is scale-invariant against pass@1 differences across models.

Chapter 5

Experimental Design

5.1 Setup

5.1.1 Inference

All inference runs on a single A100 GPU using vLLM [43] with eager execution, `bfloat16` precision, and CUDA graph compilation disabled (to make it compatible with LM-Polygraph). vLLM is initialised at 65% GPU memory utilisation, leaving room for DeBERTa in the same process.

Fair comparison across the nine LM-Polygraph estimators requires careful design. My earlier design ran Non-NLI and NLI methods in separate processes at 85% and 65% memory utilization (to speed up non-NLI), which caused 5% of greedy completions to differ between groups, despite temperature 1. The cause was not random floating point error: `gpu_memory_utilization` controls the size of the KV cache, which determines how attention is chunked. This in turn changes floating point operation ordering and produces different logits. Also, on different runs, even with the same utilization, floating point errors tend to accumulate differently. The resulting pass@1 gap affects PR-AUC and PRR because both metrics depend on the fraction of correct predictions, making results hard to compare across runs.

To avoid this, both groups run in one model instance at 65% utilisation. Greedy decoding happens once per problem, and all estimators read the resulting token sequence from a shared dependency dictionary. The four execution-based scores

adopt the same greedy completion via a `--greedy-file` argument, so all thirteen uncertainty scores share identical pass@1 by construction.

5.1.2 Prompt Construction

Each HumanEval stub is wrapped in a task instruction adapted from the DeepSeek-Coder evaluation protocol [4] and passed through each model's native chat template via `tokenizer.apply_chat_template` (DeepSeek uses `### Instruction / ### Response` delimiters, Qwen uses ChatML). The model is asked to return the completed function in a fenced code block. The body is extracted by removing the fence markers and the stub prefix.

5.1.3 Sampling

Each problem receives one greedy completion and $N = 10$ stochastic completions at temperature 1.0. These samples are reused by every method: LM-Polygraph methods read them from a `samples.jsonl` file written during inference, and the clustering methods use the same file (with the HumanEval stub to reconstruct complete function definitions). No additional model calls are needed for either clustering approach.

5.2 Functional Clustering

Test inputs are generated per problem by prompting the same model on the function signature and docstring alone, following the methodology described by [6]. The response is parsed as a JSON array of parameter-to-value dictionaries.

The reference implementation by [6] departs from this protocol on HumanEval: it extracts test inputs from the dataset's built-in `assert candidate(...)` statements instead of generating them. These are the same inputs that `evaluate_functional_correctness` uses to score correctness. This means the method observes the evaluation criterion, while every other UQ method in this comparison does not. I restored the paper's methodology by using only LLM-generated inputs, which ensures fair comparison across methods.

Each sample is concatenated with the original stub to form a complete function, then run on each input with a 10 second timeout. Two completions are merged only if they produce the same output on every input. Any completion that raises an exception or times out on any input is placed in an isolated cluster. The completion written to the output file is the greedy completion from the shared LM-Polygraph run, converted back to body-only format.

5.3 Symbolic Clustering

For each problem, cluster the N completions by querying CrossHair's `diff_behavior` for every pair. CrossHair runs with a 10-second `per_condition_timeout` and `per_path_timeout`. If a counterexample is found, the pair goes to separate clusters. If none appears before the timeout, merge the pair via union-find. It also places syntactically invalid functions in isolated clusters,rather than merging them.

A naive implementation that runs the CrossHair CLI per pair had too big of an overhead. HumanEval requires 7,380 calls per model ($\frac{N(N-1)}{2} = 45$ pairs across 164 problems), which took several hours. Most pairs timed out before reaching a CrossHair verdict, defaulting to "equivalent". I made three changes to optimize it, while preserving the same per-pair budget:

1. **In-process API.** CrossHair is called through its Python library (`crosshair.diff_behavior`) inside workers, removing per-pair subprocess startup.
2. **AST-based deduplication.** Before any pair work, completions are hashed under a normalisation that ignores whitespace, comments, and docstrings. Identical functions are merged immediately, reducing the number of distinct pairs by roughly 30-50% on HumanEval.
3. **Worker pool with hard wall-clock kill.** Pairs run in parallel across spawn-context workers. CrossHair's `per_condition_timeout` does not bound total wall-clock time, so a runaway pair (infinite loop in the completion under

analysis) is killed and the worker respawned. Killed pairs default to “equivalent”, consistent with the timeout-merging convention used elsewhere.

The 10-second per-condition budget and the rest of the methodology are unchanged from the original paper [7]. All symbolic clustering numbers reported in this thesis are produced by the optimised implementation.

5.4 Evaluation Protocol

Functional correctness is assessed with `evaluate_functional_correctness` from the HumanEval release [3]. Because this harness executes model-generated code, it runs inside a Docker container with `--network none` to block network access from generated programs. Each of the thirteen output files is evaluated independently, producing a `_results.jsonl` with a `passed` field per problem. PR-AUC and PRR are computed from the paired (uncertainty, correctness) vectors.

Chapter 6

Results

6.1 Pass@1

TABLE 4

Pass@k on HumanEval

Model	pass@1	pass@10
DeepSeek-Coder-6.7B-Instruct	0.689	0.927
Qwen2.5-Coder-7B-Instruct	0.726	0.957
DeepSeek-Coder-1.3B-Instruct	0.506	0.8537

All thirteen uncertainty methods share pass@1 per model by construction.

6.2 PR-AUC and PRR

TABLE 5

PR-AUC and PRR on DeepSeek-Coder-6.7B-Instruct

Method	PR-AUC	PRR
Functional Clustering SE	0.6068	0.8038
Functional Clustering CC	0.5935	0.8034
Lexical Similarity ROUGE-L	0.5854	0.8299
Claim-Conditioned Probability	0.5832	0.7751
Maximum Sequence Probability	0.5553	0.7608

Method	PR-AUC	PRR
DegMat Jaccard	0.5483	0.8103
SAR	0.5431	0.8325
Lexical Similarity BLEU	0.5280	0.8081
Perplexity	0.4389	0.7386
TokenSAR	0.4378	0.7373
Symbolic Clustering SE	0.4278	0.8454
Symbolic Clustering CC	0.4209	0.8376
DegMat NLI	0.3449	0.7285

On DeepSeek-6.7B, functional clustering SE and CC lead by PR-AUC (0.607 and 0.594), followed by ROUGE-L (0.585) and CCP (0.583). The PRR ordering is different: symbolic clustering SE and CC top the table (0.845 and 0.838), followed by SAR (0.833), ROUGE-L (0.830), DegMat-Jaccard (0.810), and BLEU (0.808). Functional clustering ranks first by PR-AUC but only seventh and eighth by PRR (0.80). Information-theoretic methods (CCP, MSP, Perplexity, TokenSAR) are mid-pack on both metrics. DegMat-NLI ranks last on both.

TABLE 6

PR-AUC and PRR on Qwen2.5-Coder-7B-Instruct

Method	PR-AUC	PRR
Claim-Conditioned Probability	0.6433	0.8581
Maximum Sequence Probability	0.5570	0.8586
Lexical Similarity ROUGE-L	0.4800	0.8180
Lexical Similarity BLEU	0.4566	0.8182
Perplexity	0.4521	0.8402
TokenSAR	0.4483	0.8399
DegMat Jaccard	0.4421	0.8138
SAR	0.4368	0.7996
Symbolic Clustering SE	0.4268	0.8460
Symbolic Clustering CC	0.4083	0.8455

Method	PR-AUC	PRR
Functional Clustering SE	0.4004	0.8359
DegMat NLI	0.3968	0.7971
Functional Clustering CC	0.3905	0.8352

On Qwen, CCP (0.643) and MSP (0.557) lead by PR-AUC, followed by ROUGE-L (0.480) and BLEU (0.457). By PRR, MSP (0.859) and CCP (0.858) still hold the top two, followed by symbolic clustering SE and CC (0.846), then Perplexity and TokenSAR (0.840). Functional clustering inverts its DeepSeek result: seventh by PRR (0.836) and only eleventh and thirteenth by PR-AUC (0.39–0.40). DegMat-NLI ranks near the bottom on both metrics.

TABLE 7

PR-AUC and PRR on DeepSeek-Coder-1.3B-Instruct

Method	PR-AUC	PRR
Lexical Similarity ROUGE-L	0.7131	0.6784
Claim-Conditioned Probability	0.7128	0.6446
Maximum Sequence Probability	0.6872	0.6260
DegMat Jaccard	0.6849	0.6545
SAR	0.6800	0.6468
Lexical Similarity BLEU	0.6537	0.6538
Perplexity	0.6384	0.6172
TokenSAR	0.6383	0.6220
Symbolic Clustering SE	0.5970	0.6452
Symbolic Clustering CC	0.5943	0.6438
Functional Clustering SE	0.5689	0.4801
Functional Clustering CC	0.5659	0.4804
DegMat NLI	0.5398	0.5417

On DeepSeek-1.3B, ROUGE-L (0.713) and CCP (0.713) lead by PR-AUC, followed by MSP (0.687), DegMat-Jaccard (0.685), and SAR (0.680). The lower pass@1

(50.6%) raises absolute PR-AUC across the board because failures are now the majority class. By PRR, sample-diversity methods take the top: ROUGE-L (0.678), DegMat-Jaccard (0.654), BLEU (0.654), SAR (0.647), with symbolic clustering SE and CC fifth and seventh (0.645). Functional clustering ranks last on both metrics by a wide margin (PRR 0.480, PR-AUC 0.567), reversing its DeepSeek-6.7B position.

6.3 PR-AUC and PRR Curves

To better understand the results, I have selected several PR-AUC and PRR plots.

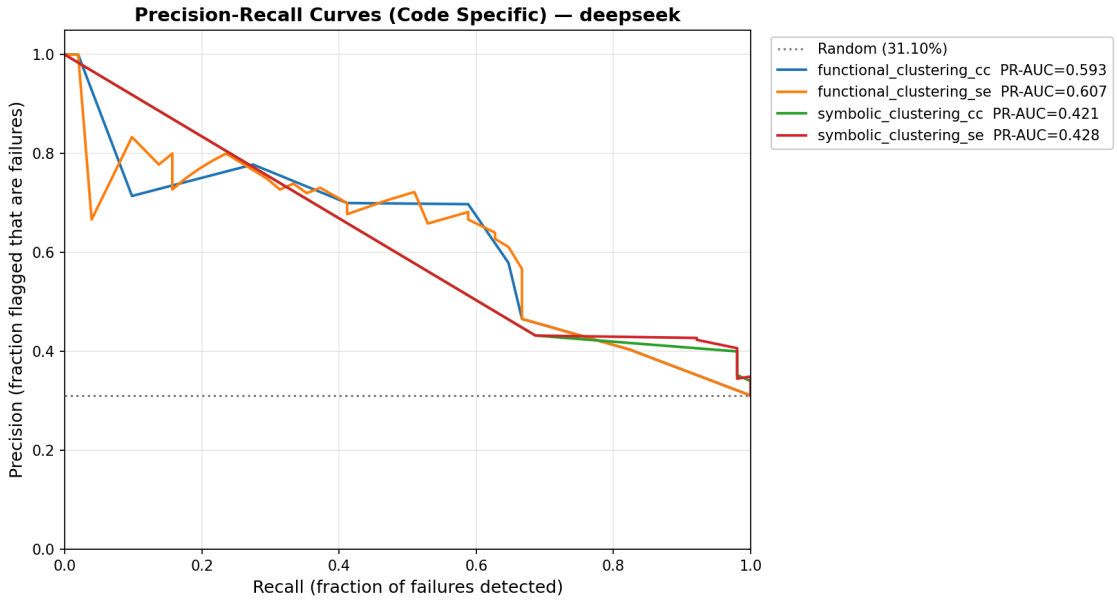

Fig. 6.1. Precision-recall curves for code-specific methods on DeepSeek-Coder-6.7B-Instruct

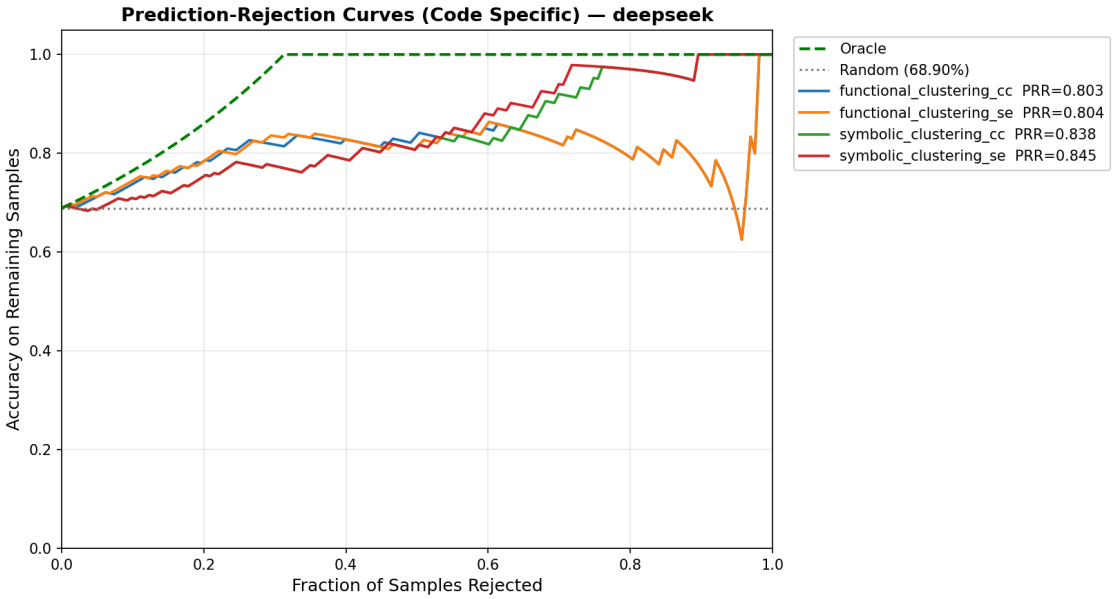

Fig. 6.2. PRR curves for code-specific methods on DeepSeek-Coder-6.7B-Instruct

On DeepSeek-6.7B, code-specific methods behave differently across metrics. In Fig. 6.1, functional clustering has higher precision over a wide recall range, leading to the best PR-AUC. In Fig. 6.2, symbolic clustering achieves higher PRR. This shows that PR-AUC and PRR favor different behaviors.

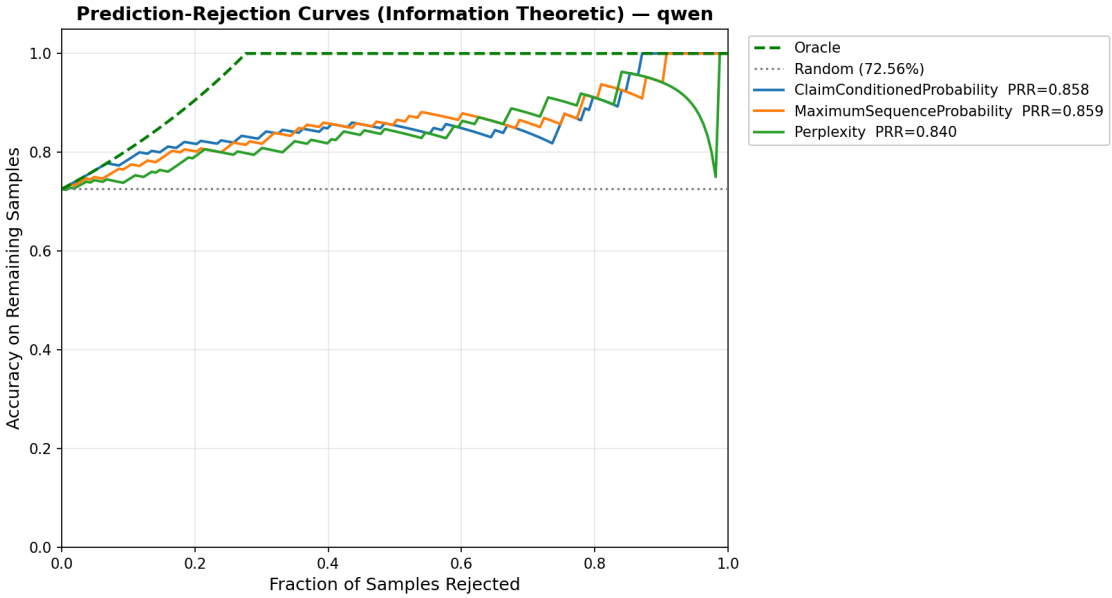

Fig. 6.3. PRR curves for information-theoretic methods on Qwen2.5-Coder-7B-Instruct

On Qwen, information-theoretic methods perform best. In Fig. 6.3, claim-conditioned probability and maximum sequence probability provide better rejection across most coverage levels, consistent with their top PRR scores.

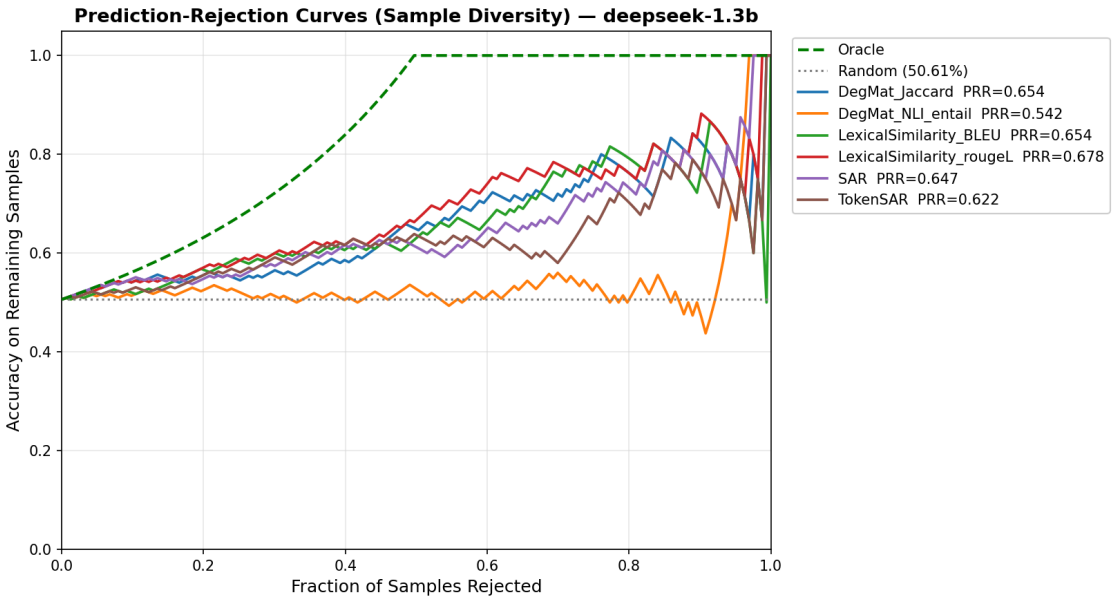

Fig. 6.4. PRR curves for diversity-based methods on DeepSeek-Coder-1.3B-Instruct

On DeepSeek-1.3B, where pass@1 is lower, diversity-based methods are stronger. In Fig. 6.4, lexical and structural diversity measures have better rejection at moderate coverage. In contrast, functional clustering performs poorly in this setting (Table 7).

Overall, the curves support three points: PR-AUC and PRR rank methods differently, no method family is best on all models, and performance depends on model quality.

6.4 Summary

No single method leads on all three models. By PR-AUC, functional clustering ranks first and second on DeepSeek-6.7B, CCP and MSP take those positions on Qwen, and ROUGE-L and CCP take them on DeepSeek-1.3B. ROUGE-L and CCP appear in the top four by PR-AUC on every model. By PRR, symbolic clustering is in the lead: top two on DeepSeek-6.7B (~0.84) and third and fourth on Qwen (~0.85). Functional

clustering inverts across model sizes: top by PR-AUC on DeepSeek-6.7B, but last on DeepSeek-1.3B by both metrics. PR-AUC and PRR rank methods differently in every table. Chapter 7.3 explains why.

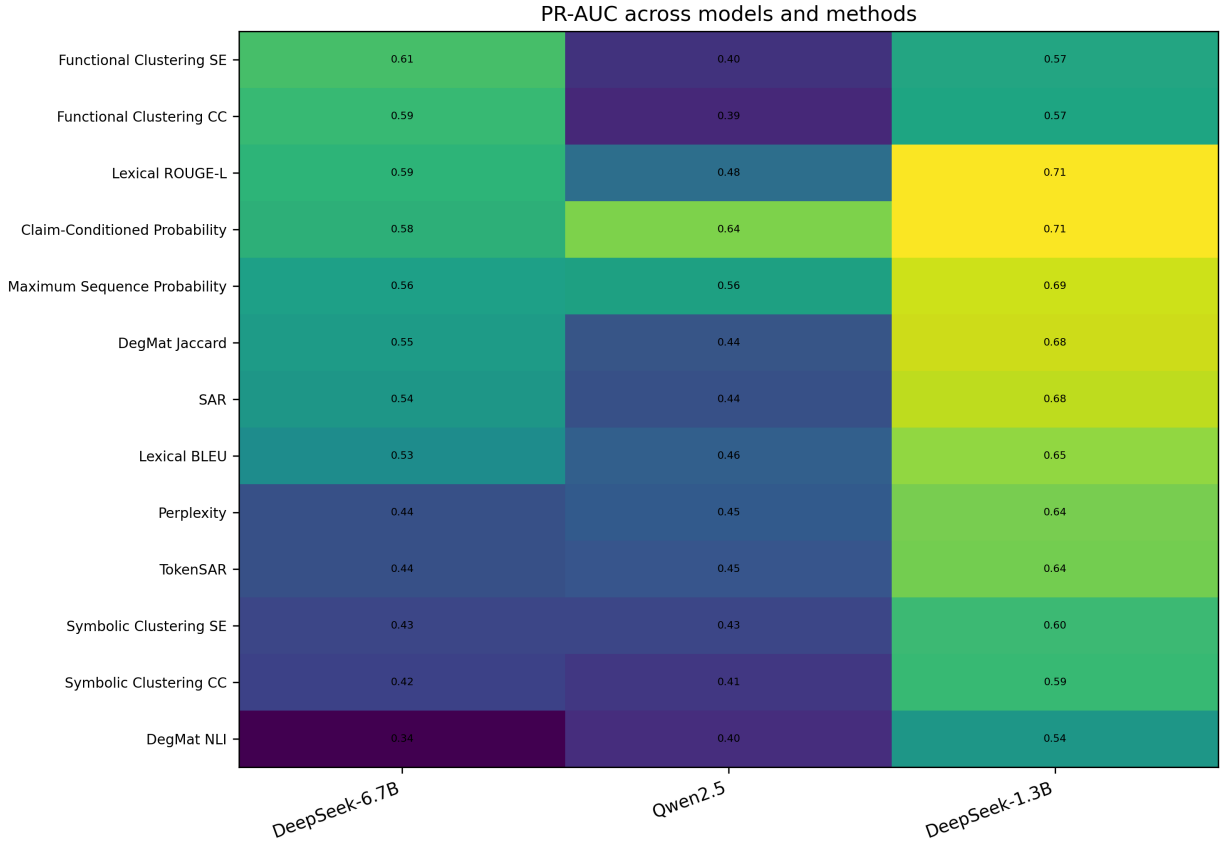

Fig. 6.5. PR-AUC heatmap across methods and models

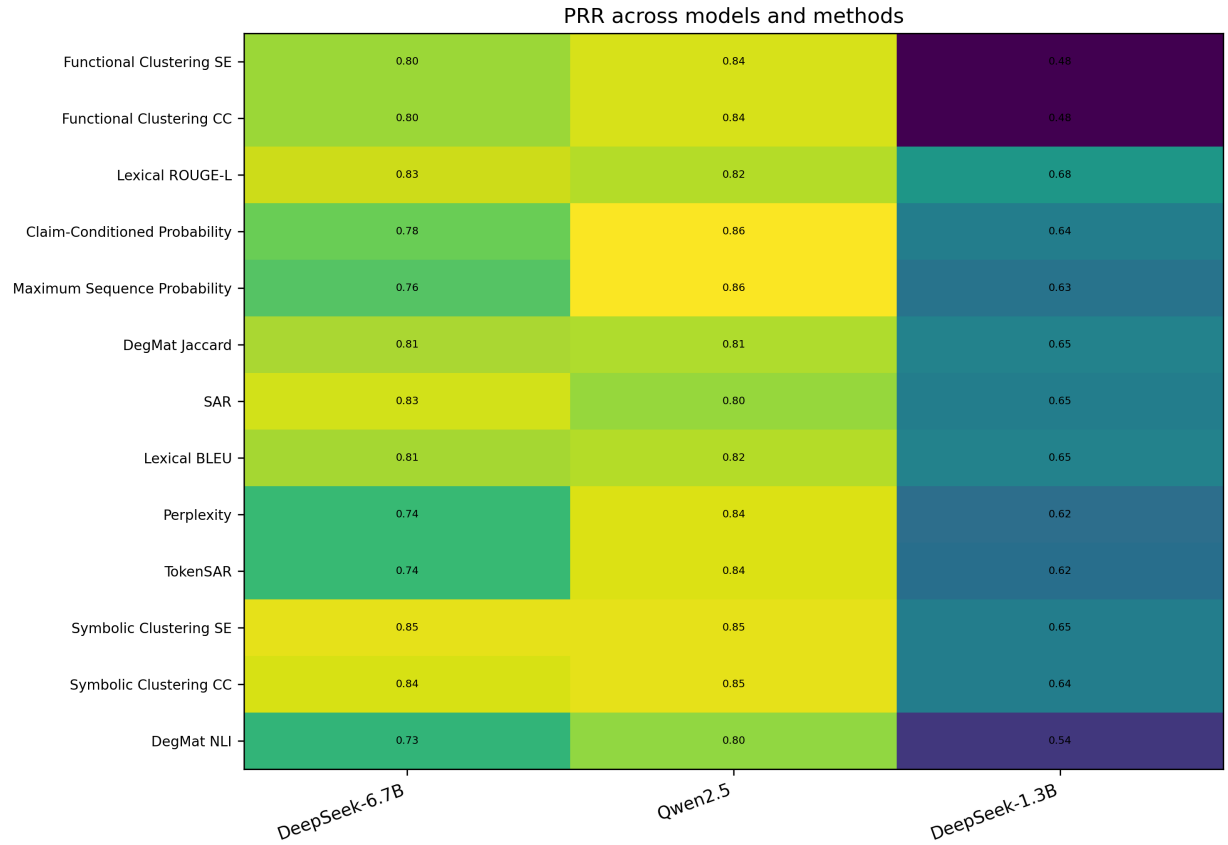

Fig. 6.6. PRR heatmap across methods and models

Chapter 7

Discussion

7.1 Symbolic Clustering Timeouts

Symbolic clustering ranks at or near the top by PRR on DeepSeek-6.7B (0.845) and Qwen (0.846), but PR-AUC stays in the lower half (~0.42 on both). The strong PRR depends on the optimised implementation described in Chapter 5.3. With the naive subprocess per each pair baseline, most pairs timed out within the 10-second budget and were merged by default. This collapsed nearly all completions into a single cluster, leaving symbolic clustering near the bottom of the table. The optimisations preserve the same timeout budget but let many more pairs reach a real CrossHair verdict, which produces the high PRR.

The gap between PRR and PR-AUC comes from symbolic clustering's score distribution. Results collapse into a small number of cluster sizes (bounded by $N = 10$), so many problems share the same score. These "ties" decrease PR-AUC at any fixed precision-recall threshold, even when the ranking is strong. Chapter 7.3 covers this in detail.

The situation on DeepSeek-1.3B is different. Symbolic clustering still works (PRR ~0.645, PR-AUC ~0.595), but does not lead. It could be that smaller models produce more diverse completions for the same problem.

7.2 Functional Clustering with LLM-Generated Test Inputs

Running with LLM-generated test inputs (as [6] describes, not as the reference implementation did) dropped functional clustering's PRR from ~0.87 to ~0.80 on DeepSeek and ~0.84 on Qwen. The magnitude of the drop shows how much of the earlier score came from test-data overlap rather than the method's own signal.

Fairly compared, functional clustering is still competitive on the larger models but no longer dominates. By PR-AUC, it ranks first and second on DeepSeek-6.7B but eleventh and thirteenth on Qwen. By PRR, it ranks seventh and eighth on both ~7B models, behind sample-diversity methods on DeepSeek and information-theoretic methods on Qwen. On DeepSeek-1.3B, the pattern inverts: functional clustering ranks last on both metrics (PRR ~0.480, PR-AUC ~0.567), four to five percentage points below every other method. Which could again be due to smaller models producing more diverse completions for the same problem, so clustering is less effective.

Therefore, execution-based clustering is most useful at model scales where the LLM is competent enough to generate informative test inputs and similar completions.

7.3 PR-AUC and PRR Disagree

The two metrics measure different aspects of an uncertainty estimator. PRR depends on the full ranking of problems by uncertainty: PRR is the area between the rejection curve and the random baseline, normalised against the oracle. PR-AUC aggregates precision across recall thresholds, so it depends on whether incorrect predictions receive higher uncertainty than correct ones, not on the order within each group.

Cluster-based estimators (functional and symbolic clustering) produce few distinct values, bounded by $N = 10$, so many problems share the same score. Whether

this hurts PRR or PR-AUC depends on how the tied scores align with correctness. If a single threshold cleanly separates correct from incorrect predictions, PR-AUC is high. If the ordering matches correctness on average but no single threshold is sharp, PRR is high while PR-AUC suffers. On DeepSeek-6.7B, functional clustering SE shows the first pattern: first by PR-AUC (0.607) but only seventh by PRR (0.804). Symbolic clustering SE shows the second: first by PRR (0.845) but eleventh by PR-AUC (0.428). Continuous estimators such as ROUGE-L produce smooth rankings with strong PRR and a flatter precision-recall trade-off.

The two metrics suit different settings. PRR fits use cases where uncertainty is used to rank candidate generations. PR-AUC fits use cases where failures are flagged at a fixed threshold, such as coding.

Chapter 8

Conclusion

Code LLMs produce hallucinations that are difficult to detect. Uncertainty quantification can flag likely failures before they reach production. However, code UQ research is fragmented: studies use different benchmarks, models, and metrics, making cross-study comparisons difficult. This thesis evaluated thirteen unsupervised uncertainty estimators on HumanEval using three open-source code LLMs: DeepSeek-Coder-6.7B-Instruct, Qwen2.5-Coder-7B-Instruct, and DeepSeek-Coder-1.3B-Instruct.

All thirteen estimators share the same greedy completions per problem on each model. This ensures that pass@1 (68.9% on DeepSeek-6.7B, 72.6% on Qwen, 50.6% on DeepSeek-1.3B) and the downstream PR-AUC and PRR reflect only uncertainty quality, not differences in model capability.

8.1 Key findings

By PR-AUC, no single method leads on all three models. Functional clustering ranks first and second on DeepSeek-6.7B (0.607, 0.594) but last on DeepSeek-1.3B and only eleventh and thirteenth on Qwen. CCP and MSP lead Qwen (0.643, 0.557), while ROUGE-L and CCP lead DeepSeek-1.3B (0.713, 0.713). ROUGE-L and CCP are the only methods in the top four by PR-AUC on all three models.

By PRR, the rewritten symbolic clustering implementation enters the top tier on the larger models: first and second on DeepSeek-6.7B (0.845, 0.838) and third and fourth on Qwen (0.846). On DeepSeek-1.3B, sample-diversity methods hold the top four (ROUGE-L, DegMat-Jaccard, BLEU, SAR). PR-AUC and PRR rank methods differently in every table. Chapter 7.3 explains why.

Model size matters. On DeepSeek-1.3B, functional clustering ranks last on both metrics, four to five percentage points below every other method. Smaller models produce more diverse completions per problem and less reliable LLM-generated test inputs. This collapses behaviorally distinct completions into the same cluster. Sample-diversity methods that read text overlap directly are robust to this shift and dominate at the smaller scale.

The compute cost of execution-based methods depends on the task: NP-hard problems or factorial-time solutions can produce extremely long runtimes. Timeouts bound this in practice, since both clustering methods fall back to merging on timeout. However, each timeout collapses potentially distinct completions into one cluster, degrading UQ quality. Compute-heavy LM-Polygraph methods have predictable cost but lack this escape valve: partial computation does not return a usable uncertainty estimate.

8.2 Contributions

- A shared-inference pipeline that gives identical pass@1 across all thirteen UQ methods on three code LLMs, so PR-AUC and PRR comparisons are not affected by differences between generation runs.
- A correction to the open source functional-clustering implementation, which used HumanEval's own assertions as test inputs instead of generating them via the LLM as [6] describes. With independent inputs, PRR drops from 0.87 to 0.80 on DeepSeek-6.7B, so part of the earlier advantage came from using the evaluation tests.
- An optimised symbolic clustering implementation.

- The first direct comparison on PR-AUC and PRR of the top LM-Polygraph estimators against execution-based clustering on a shared code benchmark. Compares two model families and two model sizes within one family, showing that the ranking depends on both family and scale.

8.3 Limitations

The study uses one benchmark (HumanEval) and three models from circa 2024 [4], [5]. All three use the same prompt, adapted from the DeepSeek-Coder HumanEval evaluation protocol [4], which may favor the DeepSeek family.

The evaluation covers only unsupervised methods, so supervised approaches such as HUQ-MD are not represented. All stochastic sampling uses $N = 10$ completions at temperature 1.0. I did not explore other temperatures or top-p settings.

I applied no post-hoc calibration to the uncertainty scores. Calibrating the scores (for example with temperature scaling on a held-out set [44]) could improve absolute PR-AUC and PRR values. However, the relative ranking of methods is expected to be less affected.

Symbolic clustering runs at one CrossHair budget (10 seconds per condition). The NLI-augmented methods (CCP, SAR, TokenSAR, DegMat-NLI) rely on DeBERTa, trained on natural-language entailment, which only approximates semantic agreement on code. Using CodeBERTa or a code-specific NLI model could improve these methods' performance.

Execution-based methods (functional and symbolic clustering) score well on HumanEval but transfer poorly to a deployed setting. HumanEval problems are short, self-contained, pure functions with explicit signatures and docstrings, which is what makes test-input generation and symbolic execution tractable. Production code is typically longer, depends on external state and side effects (I/O, databases, network), and rarely comes with the kind of typed signature that lets CrossHair reason symbolically or that lets the model generate meaningful inputs. On top of that, both methods require N extra stochastic generations per query and then either run

those generations on concrete inputs or hand them to a symbolic engine, which adds latency and a code-execution sandbox to the inference path. For real-time use, this rules out the family of methods that performs best in this benchmark.

8.4 Future work

Extending the benchmark to other datasets (MBPP [45], LiveCodeBench [46], HumanEval-X [47]) and to a wider range of model sizes and families would test how well these rankings generalise. A larger CrossHair budget could improve symbolic clustering's PR-AUC by producing more granular cluster sizes. Testing supervised methods on code would also significantly improve the benchmark. The cross-family changes between DeepSeek-6.7B and Qwen, and the breakdown of functional clustering on DeepSeek-1.3B, suggest that hybrid scores combining token-level and execution-level signals could be more robust than any single family.

A further direction is speculative uncertainty quantification: drawing the N stochastic samples from a small drafter model and consuming them with the target model's existing scoring. The cross-model PRR co-ranking in Fig. 6.6 suggests the ranking signal may survive this substitution. White-box methods that score samples under the target (CCP, SAR, TokenSAR) gain the most: cheap drafter generation replaces expensive target generation, while target scoring is a single parallel forward pass per sample. Black-box diversity methods (lexical similarity, DegMat) likely degrade, since they assume samples come from the target distribution, and execution-based methods do not benefit at all because their dominant cost is code execution rather than LLM inference. Recent work supports the direction but leaves code uncovered: [48] propose distilled drafter ensembles with a bias-variance decomposition and report strong results on GSM8K, while [49] study speculative sampling on QA and summarisation but explicitly could not benchmark it on HumanEval.

Bibliography cited

[1] Stack Overflow, “Stack Overflow Developer Survey.” [Online]. Available: https://survey.stackoverflow.co/

[2] R. Vashurin *et al.*, “Benchmarking Uncertainty Quantification Methods for Large Language Models with LM-Polygraph,” *Transactions of the Association for Computational Linguistics*, vol. 13, pp. 220–248, 2025, doi: 10.1162/tacl_a_00737.

[3] M. Chen *et al.*, “Evaluating Large Language Models Trained on Code,” 2021, [Online]. Available: https://arxiv.org/abs/2107.03374

[4] D. Guo *et al.*, “DeepSeek-Coder: When the Large Language Model Meets Programming – The Rise of Code Intelligence,” 2024, [Online]. Available: https://arxiv.org/abs/2401.14196

[5] B. Hui *et al.*, “Qwen2.5-Coder Technical Report,” 2024, [Online]. Available: https://arxiv.org/abs/2409.12186

[6] C. Ravuri and S. Amarasinghe, “Eliminating Hallucination-Induced Errors in LLM Code Generation with Functional Clustering,” 2025, [Online]. Available: https://arxiv.org/abs/2506.11021

[7] A. Sharma and C. David, “Assessing Correctness in LLM-Based Code Generation via Uncertainty Estimation,” *ArXiv*, 2025, [Online]. Available: https://arxiv.org/abs/2502.11620

[8] Z. Ji *et al.*, “Survey of Hallucination in Natural Language Generation,” *ACM Computing Surveys*, vol. 55, no. 12, pp. 1–38, Mar. 2023, doi: 10.1145/3571730.

[9] V. Agarwal *et al.*, “CodeMirage: Hallucinations in Code Generated by Large Language Models,” *ArXiv*, 2025, [Online]. Available: https://arxiv.org/abs/2408.08333

[10] F. Liu *et al.*, “Exploring and Evaluating Hallucinations in LLM-Powered Code Generation,” *ArXiv*, 2024, [Online]. Available: https://arxiv.org/abs/2404.00971

[11] Y. Tian *et al.*, “CodeHalu: investigating code hallucinations in LLMs via execution-based verification,” in *Proceedings of the Thirty-Ninth AAAI Conference on Artificial Intelligence and Thirty-Seventh Conference on Innovative Applications of Artificial Intelligence and Fifteenth Symposium on Educational Advances in Artificial Intelligence*, in AAAI'25. AAAI Press, 2025. doi: 10.1609/aaai.v39i24.34717.

[12] N. Jiang *et al.*, “Collu-Bench: A Benchmark for Predicting Language Model Hallucinations in Code,” *ArXiv*, 2024, [Online]. Available: https://arxiv.org/abs/2410.09997

[13] Z. Lin *et al.*, “Generating with Confidence: Uncertainty Quantification for Black-box Large Language Models,” *Transactions on Machine Learning Research*, 2024, [Online]. Available: https://openreview.net/forum?id=DWkJCSxKU5

[14] C. Chow, “On optimum recognition error and reject tradeoff,” *IEEE Transactions on Information Theory*, vol. 16, no. 1, pp. 41–46, 1970, doi: 10.1109/TIT.1970.1054406.

[15] Y. Geifman and R. El-Yaniv, “Selective classification for deep neural networks,” in *Proceedings of the 31st International Conference on Neural Information Processing Systems*, in NIPS'17. Curran Associates Inc., 2017, pp. 4885–4894. doi: 10.5555/3295222.3295241.

[16] Y. Gal *et al.*, “Deep Bayesian Active Learning with Image Data,” in *Proceedings of the 34th International Conference on Machine Learning*, D. Precup and

Y. W. Teh, Eds., in Proceedings of Machine Learning Research, vol. 70. PMLR, 2017, pp. 1183–1192. [Online]. Available: https://proceedings.mlr.press/v70/gal17a.html

[17] B. Lakshminarayanan *et al.*, "Simple and Scalable Predictive Uncertainty Estimation using Deep Ensembles," *ArXiv*, 2017, [Online]. Available: https://arxiv.org/abs/1612.01474

[18] C. Blundell *et al.*, "Weight Uncertainty in Neural Network," in *Proceedings of the 32nd International Conference on Machine Learning*, F. Bach and D. Blei, Eds., in Proceedings of Machine Learning Research, vol. 37. Lille, France: PMLR, 2015, pp. 1613–1622. [Online]. Available: https://proceedings.mlr.press/v37/blundell15.html

[19] A. Shelmanov *et al.*, "Active Learning for Sequence Tagging with Deep Pre-trained Models and Bayesian Uncertainty Estimates," in *Proceedings of the 16th Conference of the European Chapter of the Association for Computational Linguistics: Main Volume*, P. Merlo, J. Tiedemann, and R. Tsarfaty, Eds., Online: Association for Computational Linguistics, Apr. 2021, pp. 1698–1712. doi: 10.18653/v1/2021.eacl-main.145.

[20] A. Malinin and M. Gales, "Uncertainty Estimation in Autoregressive Structured Prediction," in *International Conference on Learning Representations*, 2021. [Online]. Available: https://openreview.net/forum?id=jN5y-zb5Q7m

[21] L. Kuhn *et al.*, "Semantic Uncertainty: Linguistic Invariances for Uncertainty Estimation in Natural Language Generation," *ArXiv*, 2023, [Online]. Available: https://arxiv.org/abs/2302.09664

[22] A. Podolskiy *et al.*, "Revisiting mahalanobis distance for transformer-based out-of-domain detection," *Proceedings of the AAAI Conference on Artificial Intelligence*, vol. 35, no. 15, pp. 13675–13682, 2021, doi: 10.1609/aaai.v35i15.17612.

[23] J. Ren *et al.*, "Out-of-Distribution Detection and Selective Generation for Conditional Language Models," in *The Eleventh International Conference on Learning Representations*, 2023. [Online]. Available: https://openreview.net/forum?id=kJUS5nD0vPB

[24] L. Smith and Y. Gal, "Understanding Measures of Uncertainty for Adversarial Example Detection," *ArXiv*, 2018, [Online]. Available: https://arxiv.org/abs/1803.08533

[25] M. Fomicheva *et al.*, "Unsupervised Quality Estimation for Neural Machine Translation," *Transactions of the Association for Computational Linguistics*, vol. 8, pp. 539–555, 2020, doi: 10.1162/tacl_a_00330.

[26] K. Lee *et al.*, "A Simple Unified Framework for Detecting Out-of-Distribution Samples and Adversarial Attacks," *ArXiv*, 2018, [Online]. Available: https://arxiv.org/abs/1807.03888

[27] K. Yoo *et al.*, "Detection of Adversarial Examples in Text Classification: Benchmark and Baseline via Robust Density Estimation," in *Findings of the Association for Computational Linguistics: ACL 2022*, S. Muresan, P. Nakov, and A. Villavicencio, Eds., Dublin, Ireland: Association for Computational Linguistics, May 2022, pp. 3656–3672. doi: 10.18653/v1/2022.findings-acl.289.

[28] S. Kadavath *et al.*, "Language Models (Mostly) Know What They Know," *ArXiv*, 2022, [Online]. Available: https://arxiv.org/abs/2207.05221

[29] T. Abbasli *et al.*, "Comparing Uncertainty Measurement and Mitigation Methods for Large Language Models: A Systematic Review.," *CoRR*, Apr. 2025, [Online]. Available: http://dblp.uni-trier.de/db/journals/corr/corr2504.html#abs-2504-18346

[30] C. Ling *et al.*, "Uncertainty Quantification for In-Context Learning of Large Language Models," in *Proceedings of the 2024 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies (Volume 1: Long Papers)*, K. Duh, H. Gomez, and S.

Bethard, Eds., Mexico City, Mexico: Association for Computational Linguistics, June 2024, pp. 3357–3370. doi: 10.18653/v1/2024.naacl-long.184.

[31] A. Malinin *et al.*, "Incorporating Uncertainty into Deep Learning for Spoken Language Assessment," in *Proceedings of the 55th Annual Meeting of the Association for Computational Linguistics (Volume 2: Short Papers)*, R. Barzilay and M.-Y. Kan, Eds., Vancouver, Canada: Association for Computational Linguistics, July 2017, pp. 45–50. doi: 10.18653/v1/P17-2008.

[32] Y. Zhu *et al.*, "Uncertainty-Guided Chain-of-Thought for Code Generation with LLMs," *ArXiv*, 2025, [Online]. Available: https://arxiv.org/abs/2503.15341

[33] K. He *et al.*, "Towards Better Code Generation: Adaptive Decoding with Uncertainty Guidance," *ArXiv*, 2025, [Online]. Available: https://arxiv.org/abs/2506.08980

[34] X. Wang *et al.*, "Compilable Neural Code Generation with Compiler Feedback," *ArXiv*, 2022, [Online]. Available: https://arxiv.org/abs/2203.05132

[35] G. Dolcetti *et al.*, "Helping LLMs Improve Code Generation Using Feedback from Testing and Static Analysis," *ArXiv*, 2025, [Online]. Available: https://arxiv.org/abs/2412.14841

[36] K. Liu *et al.*, "LLM-Powered Test Case Generation for Detecting Bugs in Plausible Programs," *ArXiv*, 2025, [Online]. Available: https://arxiv.org/abs/2404.10304

[37] E. Fadeeva *et al.*, "Fact-Checking the Output of Large Language Models via Token-Level Uncertainty Quantification," in *Findings of the Association for Computational Linguistics: ACL 2024*, L.-W. Ku, A. Martins, and V. Srikumar, Eds., Bangkok, Thailand: Association for Computational Linguistics, Aug. 2024, pp. 9367–9385. doi: 10.18653/v1/2024.findings-acl.558.

[38] J. Duan *et al.*, "Shifting Attention to Relevance: Towards the Predictive Uncertainty Quantification of Free-Form Large Language Models," in *Proceedings*

of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers), L.-W. Ku, A. Martins, and V. Srikumar, Eds., Bangkok, Thailand: Association for Computational Linguistics, Aug. 2024, pp. 5050–5063. doi: 10.18653/v1/2024.acl-long.276.

[39] C.-Y. Lin, "ROUGE: A Package for Automatic Evaluation of Summaries," in *Text Summarization Branches Out*, Barcelona, Spain: Association for Computational Linguistics, July 2004, pp. 74–81. [Online]. Available: https://aclanthology.org/W04-1013/

[40] K. Papineni *et al.*, "Bleu: a Method for Automatic Evaluation of Machine Translation," in *Proceedings of the 40th Annual Meeting of the Association for Computational Linguistics*, P. Isabelle, E. Charniak, and D. Lin, Eds., Philadelphia, Pennsylvania, USA: Association for Computational Linguistics, July 2002, pp. 311–318. doi: 10.3115/1073083.1073135.

[41] P. He *et al.*, "DeBERTa: Decoding-enhanced BERT with Disentangled Attention," 2020, [Online]. Available: https://arxiv.org/abs/2006.03654

[42] P. Schanely, "CrossHair: An analysis tool for Python that blurs the line between testing and type systems." 2024.

[43] W. Kwon *et al.*, "Efficient Memory Management for Large Language Model Serving with PagedAttention," in *Proceedings of the 29th Symposium on Operating Systems Principles*, 2023. doi: 10.1145/3600006.3613165.

[44] C. Guo *et al.*, "On calibration of modern neural networks," in *Proceedings of the 34th International Conference on Machine Learning*, in Proceedings of Machine Learning Research, vol. 70. 2017, pp. 1321–1330.

[45] J. Austin *et al.*, "Program Synthesis with Large Language Models," 2021, [Online]. Available: https://arxiv.org/abs/2108.07732

[46] N. Jain *et al.*, "LiveCodeBench: Holistic and Contamination Free Evaluation of Large Language Models for Code," 2024, [Online]. Available: https://arxiv.org/abs/2403.07974

[47] Q. Zheng *et al.*, “CodeGeeX: A Pre-Trained Model for Code Generation with Multilingual Benchmarking on HumanEval-X,” in *Proceedings of the 29th ACM SIGKDD Conference on Knowledge Discovery and Data Mining*, 2023, pp. 5673–5684.

[48] S. Park *et al.*, “Efficient Epistemic Uncertainty Estimation for Large Language Models via Knowledge Distillation,” 2026, [Online]. Available: https://arxiv.org/abs/2602.01956

[49] D. Foodeei *et al.*, “Semantic uncertainty in advanced decoding methods for LLM generation,” 2025, [Online]. Available: https://arxiv.org/abs/2506.17296